Ex No: 5.1

ENCRYPT AND DECRYPT MESSAGE USING PGP KEYS

Aim:

To encrypt and decrypt messages or files using PGP keys and GPG commands in Linux, ensuring secure data confidentiality and integrity through public key cryptography.

Procedure:

Step 1: GPG version

Check if GNU Privacy Guard (GPG) is already in your system.

$ gpg --version

```
┌──(kali㉿kali)-[~]
└─$ gpg --version
gpg (GnuPG) 2.2.46
libgcrypt 1.11.0
Copyright (C) 2024 g10 Code GmbH
License GNU GPL-3.0-or-later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Step 1(a): Install GPG (if not already installed)

$ sudo apt install gnupg

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install gnupg
gnupg is already the newest version (2.2.46-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

Step 2: Generate Key Pair

$ gpg --gen-key

```
┌──(kali㉿kali)-[~]
└─$ gpg --gen-key
gpg (GnuPG) 2.2.46; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: directory '/home/kali/.gnupg' created
gpg: keybox '/home/kali/.gnupg/pubring.kbx' created
Note: Use "gpg --full-generate-key" for a full featured key generation dialog.
```

Step 2(a): Construct a User ID

Follow prompts to enter:

Real Name: Test user (anything you wish) Email address: test@example.com

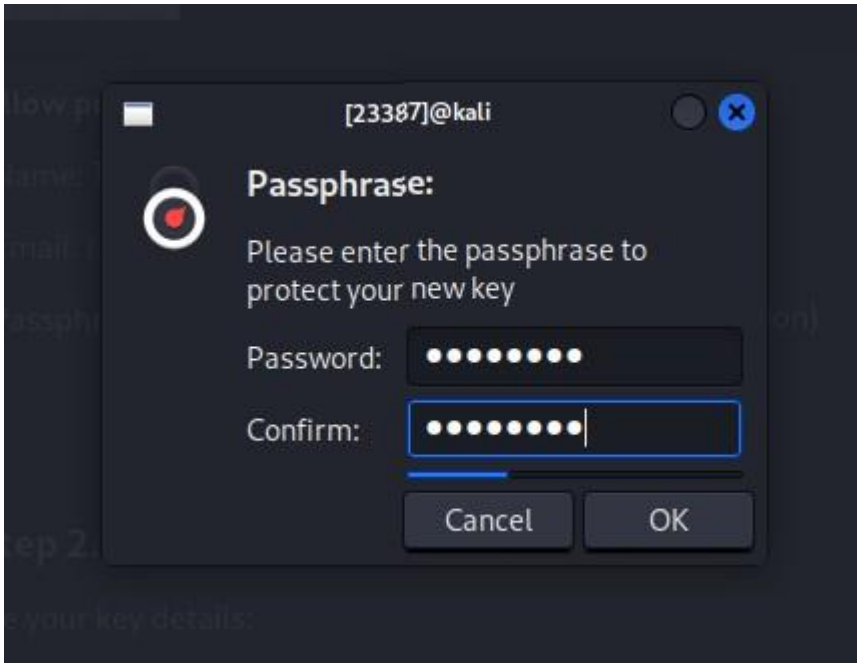
Press 'O' to confirm

```
GnuPG needs to construct a user ID to identify your key.

Real name: Test User
Email address: test@example.com
You selected this USER-ID:
    "Test User <test@example.com>"

Change (N)ame, (E)mail, or (O)kay/(Q)uit? O
```

Step 2(b): Enter a Passphrase and confirm



Step 3: Ensure if the keys are created and signed

$ gpg --list-keys

```
┌──(kali㉿kali)-[~]
└─$ gpg --list-keys

/home/kali/.gnupg/pubring.kbx
-----------------------------
pub   rsa3072 2025-07-14 [SC] [expires: 2028-07-13]
      2CDF62F0DAF436E55160F30738EF4CAF500BFB91
uid           [ultimate] Test User <test@example.com>
sub   rsa3072 2025-07-14 [E] [expires: 2028-07-13]
```

Step 4: Create a message.txt for PGP

$ nano message.txt

```
┌──(kali㉿kali)-[~]
└─$ nano message.txt
```

Step 4(a): Content on message.txt

Write your message & (i)Press Ctrl+X

(ii)Press y and

(iii)Enter to save and exit the message.txt

```
  GNU nano 8.3
This message is for testing.
```

Step 5: Encrypt a file

$ gpg -e -r “Test User” message.txt

```
┌──(kali㉿kali)-[~]
└─$ gpg -e -r "Test User" message.txt
```

This command creates message.txt.gpg which contains the encrypted version of message.txt

Step 5(a): Ensure message.txt.gpg is created

$ ls -l

```
┌──(kali㉿kali)-[~]
└─$ ls -l

total 44
drwxr-xr-x 2 kali kali 4096 Mar 26 15:32 Desktop
drwxr-xr-x 2 kali kali 4096 Mar 26 15:32 Documents
drwxr-xr-x 2 kali kali 4096 Mar 27 04:26 Downloads
-rw-rw-r-- 1 kali kali   29 Jun 29 12:47 message.txt
-rw-rw-r-- 1 kali kali  493 Jun 29 12:48 message.txt.gpg
```

$ cat m

message.txt.gpg

```
┌──(kali㉿kali)-[~]
└─$ cat message.txt.gpg
◆◆◆I◆◆◆◆
        ◆9◆!0◆a]R◆~V◆◆◆◆
◆o▒◆◆                    A.ej◆◆YK◆◆◆ ◆ɿ◆◆◆ULB~D/◆◆+b◆◆X3wf▒y◆h◆Ua◆"◆$◆◆◆◆◆.◆R▒◆◆◆Pc◆dT]%◆◆o◆ 59s@◆◆<W◆#>8
◆◆◆yD◆70u◆◆PE"◆◆◆#ʒ◆~3%b◆◆X      ◆◆◆T◆)t◆z^◆ᴝ◆b◆]◆O◆◆◆◆*3.◆9B◆◆◆◆◆I◆◆oz◆◆E◆◆◆Yr4[p~◆◆◆7◆◆dn!2◆◆q◆bN◆Sp4◆~
                                        (Y◆◆◆◆O◆d@◆*◆N◆\◆◆◆ ◆◆UE◆◆VB◆◆J◆◆lzx&ħ$◆◆4◆◆◆=◆◆W◆◆◆(50'◆◆ ◆◆
```

Its shows the content present in message.txt.gpg which is in the form of Binary encrypted data, not human-readable plaintext

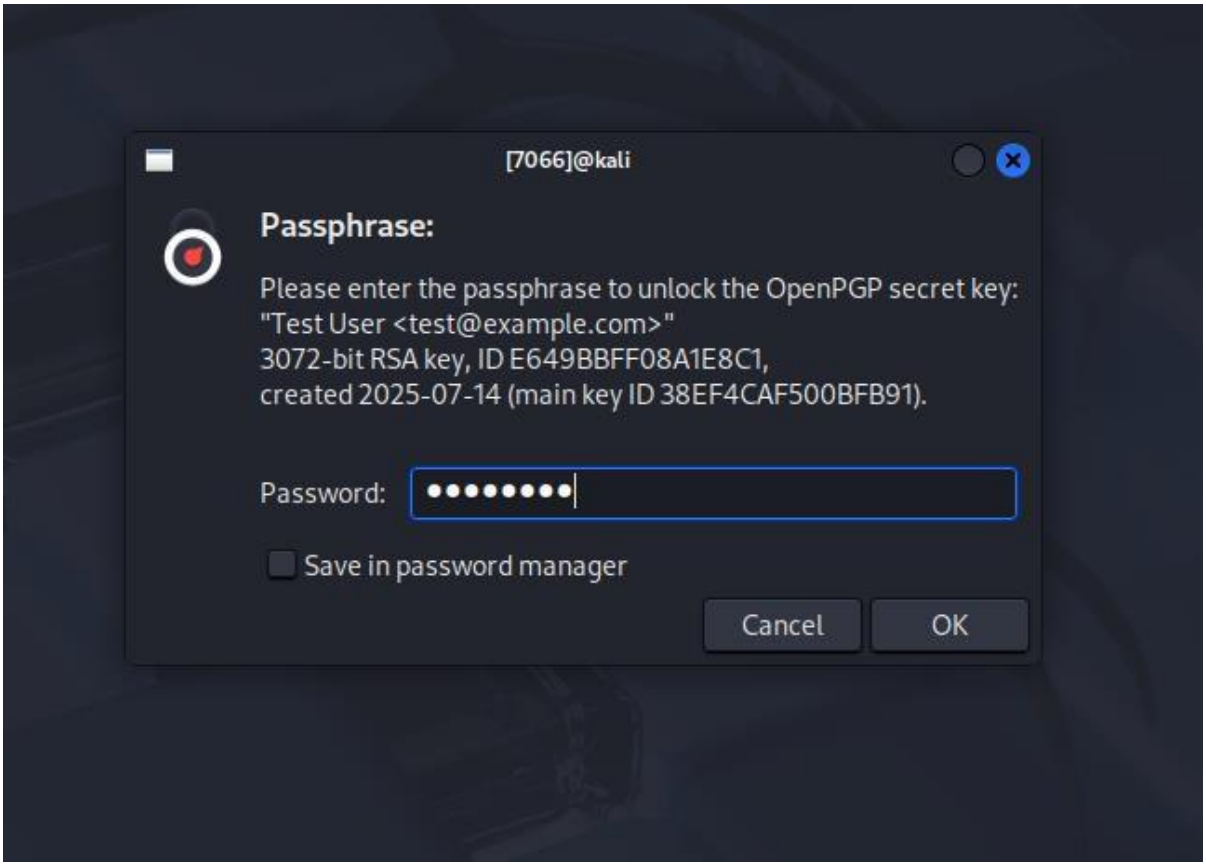
Step 6: Decrypt the file

$ gpg -d message.txt.gpg

```
┌──(kali㉿kali)-[~]
└─$ gpg -d message.txt.gpg
gpg: encrypted with 3072-bit RSA key, ID 9FCD1430765FD106, created 2025-06-29
      "Test user <test@example.com>"
This message is for testing.
```

Step 6(a): Enter your Passphrase

On Prompt: Enter your Passphrase to decrypt the message



RESULT: Thus the encryption and decryption of messages using PGP keys was successfully completed.

EX:5.b SECURE SSH ACCESS WITH FAIL2BAN AND KEY-BASED LOGIN

AIM:

To configure a Linux server to allow only key-based SSH authentication and to secure it using Fail2Ban to prevent brute-force attacks.

PROCEDURE:

Step 1: Update the System

bash

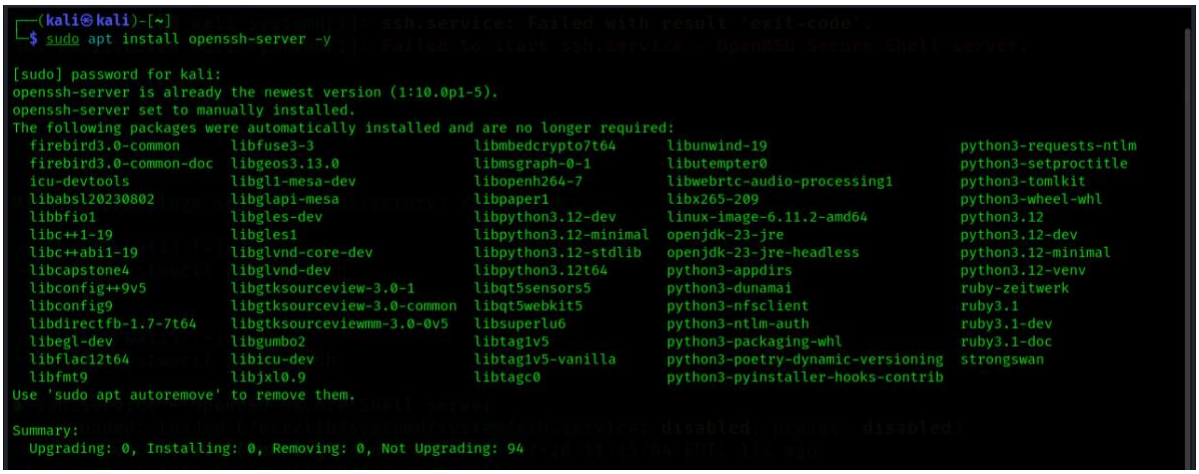
$ sudo apt update && sudo apt upgrade -y

Ensures all packages are updated for stability and security.

STEP 2: Install/OpenSSH Server (if not installed)

bash

$ sudo apt install openssh-server -y



Installs the SSH server required for remote login.

Start SSH on server:

$ sudo systemctl start ssh

$ sudo systemctl enable ssh

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl start ssh

┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink '/etc/systemd/system/sshd.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
```

Step 3: Generate SSH Key Pair (on client machine)

bash

$ ssh-keygen -t rsa -b 4096

```
┌──(kali㉿kali)-[~]
└─$ ssh-keygen -t rsa -b 4096

Generating public/private rsa key pair.
Enter file in which to save the key (/home/kali/.ssh/id_rsa): /home/kali/.ssh/id_rsa
Enter passphrase for "/home/kali/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_rsa
Your public key has been saved in /home/kali/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:WcZyZv43MhNNWzWS3KGvmOaKMaZf6GdI36oG6OKEMU0 kali@kali
The key's randomart image is:
+---[RSA 4096]----+
|             ..ooo|
|          .   oo.o|
|  E      . B   .. .|
| o        X   o.o |
|o . .    S . . o. |
|.o . . ..   .o..  |
| ...    o=o..+=.o |
| .. .   +o=+o. = . |
| ...   .o=+oo.    |
+----[SHA256]-----+
```

Creates a public-private key pair for secure SSH login.

Generates a strong public-private key pair for secure authentication.

Press Enter to accept the default location (~/.ssh/id_rsa).

For no passphrase, press Enter. Otherwise, enter your desired passphrase.

Step 4: Copy public key to server

bash

$ ssh-copy-id username@server_ip

Copies your public key to server's authorized keys.

Replace:

username → your actual username (e.g., kali, ubuntu)

server_ip → actual IP (e.g., 192.168.1.10)

To find your server ip

bash

$ ip a

```
┌──(kali㉿kali)-[~]
└─$ ssh-copy-id kali@192.168.1.11
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kali/.ssh/id_rsa.pub"
The authenticity of host '192.168.1.11 (192.168.1.11)' can't be established.
ED25519 key fingerprint is SHA256:BXxcyRHNDVtJ0tOEwFbCx4EGHALgGD+CGIRRM2a1pnI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
kali@192.168.1.11's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'kali@192.168.1.11'"
and check to make sure that only the key(s) you wanted were added.
```

STEP 5: Disable Password Login on Server

bash

$ sudo nano /etc/ssh/sshd_config

```
┌──(kali㉿kali)-[~]
└─$ sudo nano /etc/ssh/sshd_config

[sudo] password for kali:
```

Find and change or add:

PasswordAuthentication no

PermitRootLogin no

PubkeyAuthentication yes

```
  GNU nano 8.4                    /etc/ssh/sshd_config
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile     .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none

^G Help      ^O Write Out  ^F Where Is   ^K Cut      ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste    ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

Press Ctrl + O -> Enter → Ctrl + X

Restart SSH:

bash

$ sudo systemctl restart ssh

Applies the new SSH configuration settings

STEP 6: Test Key-Based Login

bash

$ ssh username@server_ip

If it logs in without a password, you're good.

STEP 7: Install Fail2Ban

bash

$ sudo apt install fail2ban -y

Installs Fail2Ban, a security tool that blocks brute-force attacks.

STEP 8: Configure Fail2Ban

bash

$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local

$ sudo nano /etc/fail2ban/jail.local

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install fail2ban -y

[sudo] password for kali:
fail2ban is already the newest version (1.1.0-8).
The following packages were automatically installed and are no longer required:
  firebird3.0-common        libgles-dev                  libqt5sensors5               python3-poetry-dynamic-versioning
  firebird3.0-common-doc    libgles1                     libqt5webkit5                python3-pyinstaller-hooks-contrib
  icu-devtools              libglvnd-core-dev            libsuperlu6                  python3-requests-ntlm
  libabsl20230802           libglvnd-dev                 libtag1v5                    python3-setproctitle
  libbfio1                  libgtksourceview-3.0-1       libtag1v5-vanilla            python3-tomlkit
  libc++1-19                libgtksourceview-3.0-common  libtagc0                     python3-wheel-whl
  libc++abi1-19             libgtksourceviewmm-3.0-0v5   libunwind-19                 python3.12
  libcapstone4              libgumbo2                    libutempter0                 python3.12-dev
  libconfig++9v5            libicu-dev                   libwebrtc-audio-processing1  python3.12-minimal
  libconfig9                libjxl0.9                    libx265-209                  python3.12-venv
  libdirectfb-1.7-7t64      libmbedcrypto7t64            linux-image-6.11.2-amd64     ruby-zeitwerk
  libegl-dev                libmsgraph-0-1               openjdk-23-jre               ruby3.1
  libflac12t64              libopenh264-7                openjdk-23-jre-headless      ruby3.1-dev
  libfmt9                   libpaper1                    python3-appdirs              ruby3.1-doc
  libfuse3-3                libpython3.12-dev            python3-dunamai              strongswan
  libgeos3.13.0             libpython3.12-minimal        python3-nfsclient
  libgl1-mesa-dev           libpython3.12-stdlib         python3-ntlm-auth
  libglapi-mesa             libpython3.12t64             python3-packaging-whl
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 94

┌──(kali㉿kali)-[~]
└─$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local

┌──(kali㉿kali)-[~]
└─$ $ sudo nano /etc/fail2ban/jail.local
```

Scroll through the file and check:

Delete the existing [DEFAULT] and [SSHD] block and replace with this.

Scroll to bottom and paste:

[DEFAULT]

bantime = 600

findtime = 600

maxretry = 3

backend = systemd

[sshd]

enabled = true

port = ssh

logpath = %(sshd_log)s

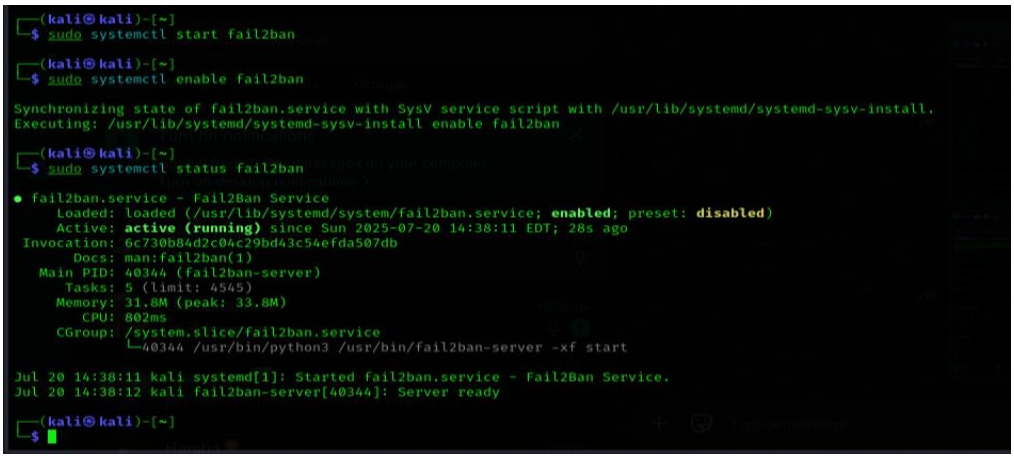
Press Ctrl + O -> Enter → Ctrl + X

STEP 9: Start and Enable Fail2Ban

bash

$ sudo systemctl start fail2ban

$ sudo systemctl enable fail2ban



STEP 10: Check Fail2Ban Status

bash

$sudo systemctl status fail2ban

$sudo fail2ban-client status

Optional – check SSH-specific stats:

bash

$sudo fail2ban-client status sshd

Result:

Thus, SSH was secured with key-based authentication and Fail2Ban, ensuring only authorized key access while blocking brute-force attacks.